

06 장 학습 과제

과제 1. **Vector2** 산술 연산

2차원 벡터를 나타내는 **Vector2** 클래스를 작성한다.

상태: `float x, y`

필수 연산자:

```
+=   벡터 덧셈 대입
-=   벡터 뺄셈 대입
*=   스칼라 곱 대입
+    벡터 덧셈
-    벡터 뺄셈과 단항 부호 반전
*    벡터와 스칼라의 곱
==   정확한 좌표 비교
<<  (x, y) 형식 출력
```

`vec2 * scalar`와 `scalar * vec2` 를 모두 지원한다. 이항 산술 연산자는 복합 대입 연산자를 재사용한다. 부동소수점 근사 비교를 위한 `NearlyEquals()`도 작성한다.

과제 2. **Money** 값 타입

금액을 나타내는 **Money** 클래스를 작성한다. 부동소수점형을 사용하지 않고 최소 화폐 단위를 `long long`으로 저장한다.

필수 기능:

- 금액 덧셈과 뺄셈
- 복합 대입 연산
- 동등 비교와 순서 비교
- 삼방향 비교 연산자
- 출력 연산자
- 음수 금액을 허용할지 정책 결정

```
Money first(1500);
Money second(2300);
Money total = first + second;
std::cout << total << '\n';
std::cout << std::boolalpha
            << (first < second) << '\n';
```

금액 출력 형식과 음수 처리 정책을 클래스 설명 주석으로 기록한다.

과제 3. **Version** 비교

주 버전, 부 버전과 패치 버전을 저장하는 **Version** 구조체를 작성한다.

```
struct Version
{
```

```

    int major;
    int minor;
    int patch;
};

```

`operator<=>`를 기본 생성하여 모든 비교 연산을 지원한다. 멤버 선언 순서가 비교 순서에 어떤 영향을 주는지 확인한다.
문자열 출력 형식은 3자를 점(dot)과 결합해서 구성한다.

1.4.2

`std::set<Version>`에 여러 버전을 저장하고 정렬 결과를 확인하는 코드는 표준 컨테이너를 학습한 뒤 확장 과제로 사용할 수 있다.

과제 4. **FixedArray** 인덱스 접근
정수 10개를 저장하는 **FixedArray** 클래스를 작성한다.

필수 기능:

```

int& operator[](std::size_t index);
const int& operator[](std::size_t index) const;
int& At(std::size_t index);
const int& At(std::size_t index) const;

```

`operator[]`는 범위를 검사하지 않고 `At()`은 범위를 검사한다. 범위를 벗어나면 `std::out_of_range`를 발생시킨다.
일반 객체와 상수 객체에서 반환 타입이 어떻게 달라지는지 확인하는 코드를 작성한다.

과제 5. **ResourceHandle**의 명시적 **bool** 변환
자원 번호와 열림 상태를 관리하는 **ResourceHandle** 클래스를 작성한다.
필수 기능:

- 기본 객체는 닫힌 상태
- 양수 자원 번호를 전달하면 열린 상태
- `Close()`를 호출하면 닫힌 상태
- `explicit operator bool()`로 유효성 확인
- `operator!()`는 작성하지 않고 `!handle`이 동작하는지 확인

```

ResourceHandle handle(10);

if (handle)
{
    std::cout << "open\n";
}

handle.Close();

```

```
if (!handle)
{
    std::cout << "closed\n";
}
```

`int value = handle;`이 허용되지 않는 이유를 설명한다.

과제 6. **Date** 스트림 입출력

연도, 월과 일을 저장하는 **Date** 클래스를 작성한다.

출력 형식

2026-08-01

입력 형식

2026 8 1

입력 연산자는 값을 임시 변수에 먼저 저장한다. 월이 1~12가 아니거나 일이 1~31이 아니면 스트림에 **failbit**를 설정하고 기존 **Date** 객체의 상태를 유지한다.

동등 비교와 삼방향 비교 연산자도 제공하여 날짜 순서를 비교한다.

과제 7. 연산자와 일반 함수 선택

게임 프로그램에서 사용할 **CharacterStats** 클래스를 설계한다.

상태는 체력, 공격력, 방어력과 이동 속도로 구성한다.

제시된 기능을 연산자로 제공할지 일반 함수로 제공할지 결정하고 이유를 작성한다.

- 두 능력치 묶음 합산
- 모든 능력치에 배율 적용
- 두 캐릭터의 전투력 비교
- 체력 회복
- 피해 적용
- 버프 병합
- 문자열 출력

연산자로 선택한 기능은 실제 코드를 작성하고, 일반 함수로 선택한 기능은 함수 이름과 매개변수 형식을 설계한다. 기본 연산자의 의미와 예상하지 못한 부작용 여부를 기준으로 판단한다.